

Solving the Heilbronn Triangle Problem using Global Optimization Methods

Amirali Modir ^{1†}, Amirhossein Monji ^{2†}, Burak Kocuk ³

^{1,2}Industrial Engineering Department, Sharif University of Technology, 1458889694, Tehran, Iran.

³Industrial Engineering Program, Sabancı University, 34956, Istanbul, Turkey.

Contributing authors: amirali.mgh@sharif.edu; amirhossein.monji@sharif.edu; burakkocuk@sabanciuniv.edu;

[†]These authors contributed equally to this work.

Abstract

We study the Heilbronn triangle problem, which involves placing n points in the unit square such that the minimum area of any triangle formed by these points is maximized. A straightforward maximin formulation of this problem is highly non-linear and non-convex due to the existence of bilinear terms and absolute value equations. We propose two mixed-integer quadratically constrained programming (MIQCP) and one QCP formulation, which can be readily solved by any global optimization solver. We develop several formulation enhancements in the form of bound tightening and symmetry breaking inequalities that are prevalent in the global optimization literature in addition to other enhancements that exploit the problem structure. With the help of these enhancements, our models reproduce proven optimal values for instances up to $n = 8$ points with certified optimality in the order of seconds. In the case of $n = 9$ points, for which no analytical proof is known, we establish a certified optimal value by a computational effort of one day. This is a significant improvement over the previous benchmark established in 31 days of computations by Chen et al. (2017).

Keywords: Heilbronn triangle problem; global optimization; mixed-integer quadratically constrained programming; mixed-integer linear programming; discretization

1 Introduction

The Heilbronn triangle problem involves placing n points ($n > 2$) in the unit square $[0, 1] \times [0, 1]$ such that the minimum area of any triangle formed by these points is maximized. More precisely, denoting the coordinates of point i as $(x_i, y_i) \in [0, 1] \times [0, 1]$ for $i = 1, \dots, n$, our aim is to solve the following optimization problem:

$$H_n^* := \max_{(x,y) \in [0,1]^n \times [0,1]^n} \min_{1 \leq i < j < k \leq n} \left\{ \frac{1}{2} |x_i(y_j - y_k) - x_j(y_i - y_k) + x_k(y_i - y_j)| \right\}. \quad (1)$$

Notice that the expression $\frac{1}{2} |x_i(y_j - y_k) - x_j(y_i - y_k) + x_k(y_i - y_j)|$ gives the area of the triangle formed by points i, j and k . We remark that problem (1) is highly non-convex due to the existence of the bilinear terms as well as the absolute value function. For small n values such as 3, 4 or 5, the optimal configurations are trivial to obtain (see Figure 1 for an illustration). However, for slightly larger values of n , the Heilbronn triangle problem becomes notoriously difficult to solve. In this paper, our aim is to solve this problem up to $n = 9$ points using global optimization methods.

The study of Heilbronn's triangle problem has evolved through several distinct lines of inquiry, from asymptotic bounds to exact computational proofs. Early work focused on worst-case analytic bounds, a field initiated by Roth in 1951, who developed a machinery for proving upper bounds on H_n^* and established that $H_n^* \ll \frac{1}{n\sqrt{\log \log n}}$ [1] (here, $A \ll B$ means that $A \leq cB$ for an absolute constant c). He later sharpened this

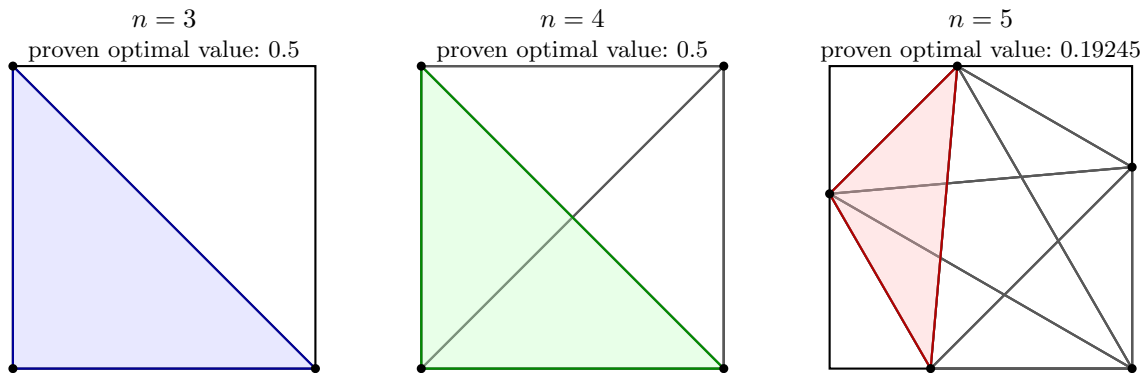


Fig. 1: Optimal point placements for $n = 3$ to $n = 5$ for the Heilbronn triangle problem.

result by obtaining bounds of the form $H_n^* \ll n^{-\mu+\varepsilon}$ for any $\varepsilon > 0$, where the exponent μ was improved from approximately 1.1056 [2, 3] to 1.117 [4]. Building on this framework, the authors of [5] improved this bound, effectively increasing the exponent to $\mu = 8/7$, by proving $H_n^* \ll n^{-8/7+\varepsilon}$. Later, the same authors established a complementary lower bound of $H_n^* \gtrsim (\log n)/n^2$ in [6], disproving Heilbronn’s original conjecture that $H_n^* = \Theta(1/n^2)$.

Complementing the asymptotic theory, researchers have also employed heuristic and constructive methods to find specific point placements that yield strong lower bounds [7–9]. Distinct from these heuristic approaches, exact results for specific small values of n have been proven using analytical and computer-algebra methods. In particular, the precise values for $n = 5$ and $n = 6$ were established in [10], and more recently for $n = 7$ and $n = 8$, proofs leveraging symbolic computation and automated deduction were used to certify optimal configurations [11, 12]. Note that the deterministic worst-case value H_n contrasts sharply with the average case for random points, where the smallest triangle area is expected to be much smaller, on the order of $\Theta(1/n^3)$ [13, 14].

For instances such as $n = 9$, where exact analytical proofs have remained elusive, high-throughput discretization and parallel enumeration have produced extremely tight numerical bounds. For instance, using a specialized grid search method accelerated by GPGPU computing, the authors of [15] obtained the following bounds: $0.054875999 < H_9^* < 0.054878314$. They established this dual bound by employing a recursive branch-and-bound algorithm over increasingly fine grid discretizations of the unit square, where local upper bounds were calculated for each rectangle configuration to rigorously prune the search space [15]. In contrast, the approach adopted in this work frames the problem as a maximin mathematical program, leveraging deterministic global optimization techniques—specifically Mixed-Integer Nonlinear Programming (MINLP)—to achieve certified results. Rather than relying on exhaustive enumeration, our methodology constructs optimality certificates by integrating tight continuous relaxations with adaptive discretization strategies. This distinction is crucial since, while analytical proofs for $n \geq 9$ remain elusive, existing computational methods often focus on heuristic lower bounds without providing the matching upper bounds necessary for certification. Consequently, rigorous global optimization frameworks specifically tailored for certifying Heilbronn numbers have remained largely underdeveloped in literature.

In this paper, we apply global optimization methods to the Heilbronn triangle problem. In particular, we develop three novel mathematical programming models to solve this problem with global optimization solvers Gurobi [16] and BARON [17]. The difference between these models lies in the way they handle the non-convex relations in problem (1) as detailed in Section 2.2. Two of these models belong to the class of mixed-integer quadratically constrained programming (MIQCP) whereas the other model belongs to the class of QCP. We propose several enhancements to strengthen these formulations by exploiting problem structure and using well-known techniques from global optimization literature such as bound tightening and symmetry breaking. We carry out extensive computations to decide the best performing model as well as the best performing solver in addition to the most effective combination of enhancements. Our rigorous analysis has enabled us to certify the exact value $H_9^* = 0.0548767$ (to within solver tolerances) for the first time in the literature, closing the optimality gap that remained in previous computational studies.

We now list our main contributions in this paper:

- *Novel formulations.* We develop three novel formulations for the Heilbronn triangle problem that can be used to solve any instance of size n . These formulations belong to the class of MIQCP or QCP, hence, any global optimization solver supporting these classes of problems is readily applicable to solve the Heilbronn triangle problem.

- *Formulation enhancements.* We propose several enhancements to strengthen the proposed formulations. Some of these enhancements are *universal*, in the sense that they are applicable to any instance with an arbitrary number of points n . Such enhancements involve bound tightening and symmetry breaking inequalities, commonly used in the global optimization literature. We also propose instance-specific enhancements that work for a particular value of n . For example, we show that for the nine-point problem, eight of these points must be located close to the boundary of the unit box (see Proposition 5 for a precise statement).
- *Certified solutions by instance size.* Using our novel formulations and utilizing our enhancements, we are able to efficiently recover the proven global optima for $n = 3, \dots, 8$ within solver tolerances in the order of seconds. We then certify the global optimum for $n = 9$ as $H_9^* = 0.0548767$ by establishing a dual bound that matches the smallest area of a feasible configuration. This is a significant improvement over the previous state-of-the-art [15], which employed a specialized GPGPU-based grid search over 31 days to establish a narrow bracket ($0.054875999 < H_9^* < 0.054878314$), whereas our method provides a certified value within one-day of computational effort. Finally, for $n = 10$, we also provide, without an optimality certificate, a configuration that matches the best-known result. Our model found a placement yielding a lower bound of $H_{10}^* \geq 0.0465369$ in approximately 800 seconds, which is consistent with the best empirical value of 0.046537 previously reported [8].

The remainder of this paper is organized as follows. Section 2 formally introduces our three novel formulations for the Heilbronn triangle problem. Section 3 proposes three groups of enhancements that significantly strengthen these formulations. Section 4 reports the results of our extensive computational study, which involves comparing formulations and solvers together with an assessment of the impact of each enhancement group. Finally, Section 5 summarizes our main findings and discusses directions for future work.

2 Problem Formulation and Solution Approaches

In this section, we formally define the Heilbronn triangle problem and present three novel mathematical programming formulations that can be solved using global optimization solvers.

2.1 Problem Formulation

We first reformulate problem (1) as

$$\max_{x,y,H,S} H_n \tag{2}$$

$$\text{s.t. } |S_{ijk}| \geq H_n \quad 1 \leq i < j < k \leq n \tag{3}$$

$$S_{ijk} = \frac{1}{2}[x_i(y_j - y_k) - x_j(y_i - y_k) + x_k(y_i - y_j)] \quad 1 \leq i < j < k \leq n \tag{4}$$

$$-\frac{1}{2} \leq S_{ijk} \leq \frac{1}{2} \quad 1 \leq i < j < k \leq n \tag{5}$$

$$0 \leq x_i, y_i \leq 1 \quad 1 \leq i \leq n \tag{6}$$

$$\underline{H}_n \leq H_n \leq \overline{H}_n, \tag{7}$$

where variable S_{ijk} represents the *signed area* of the triangle with corner points i, j and k , and H_n is the objective function variable representing the area of the smallest triangle of the Heilbronn triangle problem with n points. Note that bounds on variable S_{ijk} stated in constraint (5) follow from the definition of this variable given in constraint (4) and the fact that $0 \leq x_i, y_i \leq 1$ as given in constraint (6). The procedure to obtain bounds on variable H_n is explained later in Section 3.1.1. While constraint (7) might appear redundant for the problem definition, these *a priori* bounds are computationally critical. In particular, the upper bound $\overline{H}_n$ is essential for constructing a tight linearization of the absolute value constraint (3) (as detailed in Section 2.2.1), and the lower bound $\underline{H}_n$ allows the solver to effectively prune the branch-and-bound search tree.

Notice that the above formulation is highly non-linear and non-convex due to the reverse convex constraint (3) and the bilinear constraint (4). In the sequel, we propose three novel approaches to obtain MIQCP or QCP formulations so that available global optimization solvers can be directly used.

2.2 Solution Approaches

The primary challenge in formulating the Heilbronn triangle problem for global optimization solvers is handling the non-convex absolute value function required for the standard area calculation. In this section, we present three novel mathematical programming approaches, each designed to address this core difficulty in a different way.

2.2.1 Approach 1

The main idea of this approach is to rewrite the non-convex constraint $H_n \leq |S_{ijk}|$ as a pair of linear inequalities with the help of additional binary variables. To achieve this, a binary variable b_{ijk} is introduced for each triangle to disjunctively model the sign of the area S_{ijk} . In particular, the desired relationship is established using the following set of inequalities:

$$(1 - b_{ijk}) \left(\overline{H}_n + \frac{1}{2} \right) + S_{ijk} \geq H_n \quad 1 \leq i < j < k \leq n \quad (8a)$$

$$b_{ijk} \left(\overline{H}_n + \frac{1}{2} \right) - S_{ijk} \geq H_n \quad 1 \leq i < j < k \leq n \quad (8b)$$

$$-\frac{1}{2} \leq S_{ijk} - \left(\frac{1}{2} + \underline{H}_n \right) b_{ijk} \leq -\underline{H}_n \quad 1 \leq i < j < k \leq n \quad (8c)$$

$$b_{ijk} \in \{0, 1\} \quad 1 \leq i < j < k \leq n. \quad (8d)$$

The first pair of inequalities in (8) encodes the non-convex constraint $H_n \leq |S_{ijk}|$ via binary variables while the second pair is also valid and is included in the implementation to strengthen the formulation.

Then, we obtain the following equivalent mixed-integer quadratically constrained program (MIQCP):

$$\max_{x,y,H,S,b} \{H_n : (4) - (8)\}.$$

To further structure the model, we isolate the bilinear terms from the area formula by defining new variables w_{ij} :

$$w_{ij} = x_i y_j \quad 1 \leq i, j \leq n, \quad (9)$$

This results in a linear expression for the signed area in terms of these new variables; and rewriting (4) as

$$S_{ijk} = \frac{1}{2} [(w_{ij} - w_{ik}) - (w_{ji} - w_{jk}) + (w_{ki} - w_{kj})] \quad 1 \leq i < j < k \leq n. \quad (10)$$

The resulting MIQCP formulation, which is the form we use in our implementation, is as follows:

$$\max_{x,y,w,H,S,b} \{H : (5) - (10)\}.$$

This model is an MIQCP because it contains both the binary variables b_{ijk} and the non-convex quadratic equality constraints in (9).

2.2.2 Approach 2

The main idea of this approach is to avoid the binary variables of Section 2.2.1 by using a non-convex quadratic formulation. Instead of linearizing the absolute value, we introduce a new continuous, non-negative variable, $U_{ijk} \geq 0$, to represent the (unsigned) area of the triangle formed by points i , j , and k . The non-convex constraint $H_n \leq |S_{ijk}|$ from (3) is then replaced by the following pair of constraints:

$$U_{ijk} \geq H_n \quad 1 \leq i < j < k \leq n \quad (11)$$

$$S_{ijk}^2 = U_{ijk}^2 \quad 1 \leq i < j < k \leq n. \quad (12)$$

Since $U_{ijk} \geq 0$ is implied by the fact that $H_n \geq 0$, the quadratic equality (12) is equivalent to $U_{ijk} = |S_{ijk}|$. This substitution transforms the model into a QCP that contains no binary variables.

As in Approach 1, we use the intermediate w_{ij} variables defined in (9) and the linear expression for S_{ijk} from (10). The resulting QCP formulation, which matches our implementation, is as follows:

$$\max_{x,y,w,H,S,U} \{H_n : (5) - (7), (9), (10), (11), (12)\}.$$

Notice that this formulation is a non-convex QCP due to quadratic equality constraints in (9) and (12).

2.2.3 Approach 3

The main idea of this approach is to discretize the continuous variable $x_i \in [0, 1]$ in order to obtain a formulation with a strong relaxation. In particular, for a positive integer P , we discretize variable x_i as

$$x_i = \sum_{p=1}^P 2^{-p} \xi_{ip} + \epsilon_i, \quad \text{where } \xi_{ip} \in \{0, 1\} \text{ and } \epsilon_i \in [0, 2^{-P}].$$

Then, we have

$$w_{ij} = x_i y_j = \sum_{p=1}^P 2^{-p} \xi_{ip} y_j + \epsilon_i y_j, \quad 1 \leq i, j \leq n.$$

Observe that we rewrite the ‘continuous-times-continuous’ term $w_{ij} = x_i y_j$, as a sum of ‘binary-times-continuous’ products $\xi_{ip} y_j$ and another ‘continuous-times-continuous’ term $\epsilon_i y_j$, which involves a non-negative variable ϵ_i with a small upper bound. This new structure is highly advantageous because ‘binary-times-continuous’ products can be linearized exactly using standard McCormick envelopes defined in Fact 1 and ‘continuous-times-continuous’ product is easier to handle when at least one variable has a small range.

Fact 1. *The convex hull of the set $\{(x, y) \in [\underline{x}, \bar{x}] \times [\underline{y}, \bar{y}] : w = xy\}$ is given by the McCormick envelopes [18]:*

$$\mathcal{M}(\underline{x}, \bar{x}; \underline{y}, \bar{y}) = \{(x, y, w) : w \geq \underline{x}y + x\underline{y} - \underline{x}\underline{y}, w \geq \bar{x}y + x\bar{y} - \bar{x}\bar{y}, w \leq \bar{x}y + \underline{x}\bar{y} - \bar{x}\underline{y}, w \leq \underline{x}y + x\bar{y} - \underline{x}\bar{y}\}.$$

This reformulation is expressed through the following set of constraints, where new variables ϕ_{ipj} and ω_{ij} are introduced to represent the product terms:

$$w_{ij} = \sum_{p=1}^P 2^{-p} \phi_{ipj} + \omega_{ij} \quad 1 \leq i, j \leq n \quad (13a)$$

$$(\xi_{ip}, y_j, \phi_{ipj}) \in \mathcal{M}(0, 1; 0, 1) \quad 1 \leq i, j \leq n, p = 1, \dots, P \quad (13b)$$

$$\xi_{ip} \in \{0, 1\} \quad 1 \leq i \leq n, p = 1, \dots, P \quad (13c)$$

$$\omega_{ij} = \epsilon_i y_j \quad 1 \leq i, j \leq n \quad (13d)$$

$$0 \leq \epsilon_i \leq 2^{-P} \quad 1 \leq i \leq n. \quad (13e)$$

The resulting MIQCP formulation is as follows:

$$\max_{x, y, w, H, S, b, \xi, \phi, \omega} \{H : (5) - (8), (10), (13)\}.$$

While this formulation is exact, it still contains the non-convex quadratic constraint (13d). To obtain a model solvable with MILP techniques, we relax this final non-convex term. This gives rise to a Mixed-Integer Linear Programming (MILP) relaxation, which we use in our computational study in Section 4.2.1. This MILP relaxation is formulated as follows:

$$\max_{x, y, w, H, S, b, \xi, \phi, \omega} \{H : (5) - (8), (10), (13a) - (13c), (\epsilon_i, y_j, \omega_{ij}) \in \mathcal{M}(0, 2^{-P}; 0, 1) \ 1 \leq i, j \leq n\}.$$

This approach follows a line of papers that has successfully implemented similar ideas on related problems, including the pooling problem [19–21] and the circle packing problem [22, 23], among others.

Note that all the formulations introduced in this section are challenging to solve in practice. In order to accelerate the solution process, we introduce several enhancements in the next section.

3 Enhancements

We introduce several enhancements to strengthen the formulations introduced before and shorten the overall computation time. We categorize these enhancements into thematic categories, hereafter termed as *enhancement groups*, and assess the impact of each group as a whole rather than of every individual enhancement as detailed below.

3.1 First Group: Bound Tightening and Symmetry Breaking

3.1.1 Bounds for the Objective Value H_n

Note that any feasible solution yields a valid lower bound for a maximization problem and it is trivial to generate feasible n -point configurations in our case. To this end, we sample 10^6 i.i.d. n -point configurations from the unit square $[0, 1]^2$, and we set the lower bound for H_n as

$$\underline{H}_n = \max\{\text{minimum-triangle-area of the sample}\}.$$

To obtain an upper bound on H_n , we rely on the value of H_{n-1}^* as formalized below:

Proposition 1. *For $n > 3$, the optimal minimum-triangle area is non-increasing in n ; in particular,*

$$H_n^* \leq H_{n-1}^*.$$

Proof. Consider an optimal solution of the n -point problem given as (x_i^*, y_i^*) , $i = 1, \dots, n$ with the minimum area of H_n^* . Choose a point (x_j^*, y_j^*) among these n points such that the area of the smallest triangle formed by the remaining $n-1$ points is still H_n^* (note that such a point exists: either all triangles formed by n points have area of H_n^* , in which case any point can be chosen; or there exists a point which is not a corner of the smallest triangle that can be chosen). Clearly, the points (x_i^*, y_i^*) , $i = 1, \dots, n, i \neq j$ is a feasible solution of the $(n-1)$ -point problem with objective function value of H_n^* . Therefore, we conclude that $H_n^* \leq H_{n-1}^*$. $\square$

Due to Proposition 1, we set $\bar{H}_n = H_{n-1}^*$.

3.1.2 Symmetry Breaking

Similar to other geometry optimization problems, the Heilbronn triangle problem has many symmetrical solutions in the sense that by relabeling the points corresponding to a certain solution, we can obtain solutions that are fundamentally the same. In order to prevent this issue, which causes the optimization solvers to stall, we propose several symmetry breaking constraints given in the next proposition.

Proposition 2. *There exists an optimal solution $\{(x_k, y_k)\}_{k=1}^n$ to the Heilbronn problem satisfying the following inequalities:*

- (i) $y_1 \leq y_2 \leq \dots \leq y_n$.
- (ii) $w_{i1} \leq w_{i2} \leq \dots \leq w_{in}$.
- (iii) $x_2 \geq x_1$ and $x_1 \leq \frac{1}{2}$.

Proof. In the proof, we will use the fact that the feasible set and objective function are invariant under point relabelings and under the vertical reflection $x \mapsto 1-x$. (i) This result follows by sorting the set $\{y_k\}_{k=1}^n$ and relabeling accordingly to obtain $y_1 \leq \dots \leq y_n$. (ii) This result is a consequence of Item (i) and the definition of $w_{ij} = x_i y_j$. (iii) This result is verified as follows: If $x_2 < x_1$, then we swap labels 1 and 2. If $x_1 > \frac{1}{2}$, we reflect the entire configuration across the line $x = \frac{1}{2}$ via $(x, y) \mapsto (1-x, y)$ (and, if needed, re-swap labels 1 and 2 to keep $x_2 \geq x_1$). Both relabeling and reflection preserve all triangle areas and the constraints of the square, hence feasibility and the objective value are preserved. $\square$

3.2 Second Group: Points on the Boundary

As the Heilbronn triangle problem seeks to locate points in a way that the area of the smallest triangle is maximized, it is intuitive to expect some points to be located at the edges of the unit square $[0, 1]^2$. The second group of enhancements formalizes this intuition.

3.2.1 At Least One Point on Each Edge ($n \geq 4$)

Proposition 3. *For any optimal solution of the Heilbronn triangle problem with at least four points, each edge of the unit square $[0, 1]^2$ contains at least one point.*

Proof. Let $a = \min_i x_i$, $b = \max_i x_i$, $c = \min_i y_i$, and $d = \max_i y_i$. If $[a, b] \times [c, d]$ is a proper subrectangle of $[0, 1]^2$, then $(b-a)(d-c) < 1$. Consider the affine map $T : [a, b] \times [c, d] \rightarrow [0, 1]^2$,

$$T(x, y) = \left(\frac{x-a}{b-a}, \frac{y-c}{d-c} \right).$$

For any triangle Δ determined by three of the points, $\text{area}(T(\Delta)) = \text{area}(\Delta)/((b-a)(d-c))$. Since $(b-a)(d-c) < 1$, the area of each triangle strictly increases under T , so the minimum triangle area strictly

increases as well—contradicting optimality. Hence, we obtain $(b - a)(d - c) = 1$, which implies that $a = 0$, $b = 1$, $c = 0$, and $d = 1$. $\square$

The following corollary is a consequence of Proposition 3.

Corollary 1. *For the Heilbronn triangle problem with at least four points, there exists an optimal solution that satisfies the following constraints:*

- (i) $y_1 = 0$ and $y_n = 1$.
- (ii) $w_{i1} = 0$ and $w_{in} = x_i$ for $i = 1, \dots, n$.
- (iii) For some binary variables $c_{1i}, c_{2i} \in \{0, 1\}$, $i = 1, \dots, n$,

$$1 \leq \sum_{i=1}^n c_{1i} \leq 2, \quad x_i \leq 1 - c_{1i}, \quad i = 1, \dots, n \quad (14a)$$

$$1 \leq \sum_{i=1}^n c_{2i} \leq 2, \quad x_i \geq c_{2i}, \quad i = 1, \dots, n. \quad (14b)$$

Proof. Recall that each edge of the unit square $[0, 1]^2$ must contain at least one point due to Proposition 3. Notice that equations in Item (i) follow from Proposition 2(i) and guarantee that edges $y = 0$ and $y = 1$ of the box $[0, 1]^2$ contain at least one point. Similarly, equations in Item (ii) follow from Proposition 2(ii).

Let us now look at equations in Item (iii). In fact, inequalities (14a) (resp. inequalities (14b)) guarantee that at least one point is located at the edge $x = 0$ (resp. $x = 1$) of the box $[0, 1]^2$. Moreover, the upper bound of two makes sure that no more than two points are located in these edges, since otherwise, there would be three collinear points. $\square$

3.2.2 Two Points on Some Edge ($n \in \{7, 8\}$)

For some n values, optimal configurations place multiple points on the boundary of the unit square. In particular, for 7-point and 8-point problems, the following result is proven in references [11] and [12], respectively.

Proposition 4. *For any optimal solution of the Heilbronn triangle problem with seven or eight points, there exists an edge of the unit square $[0, 1]^2$ that contains exactly two points.*

Remark 1. *While this property is proven in the literature for $n = 7$ [11] and $n = 8$ [12], a corresponding mathematical proof for $n = 9$ is not known. As detailed in Section 3.2.3, we are able to computationally certify a related near-boundary property for $n = 9$.*

The following corollary is a consequence of Proposition 2 and Proposition 4.

Corollary 2. *For the Heilbronn triangle problem with seven or eight points, there exists an optimal solution that satisfies the following constraints:*

- (i) $y_1 = y_2 = 0$.
- (ii) $w_{i1} = w_{i2} = 0$ for $i = 1, \dots, n$.

3.2.3 Approximately Eight Boundary Points ($n = 9$)

In all of our experiments related to the Heilbronn triangle problem with at least nine points, we observe that eight points are placed on the boundary of the unit square $[0, 1]^2$ across all best configurations. The same pattern was also seen in all best-known configurations for $n \geq 9$ reported in the literature. Hence, we have come up with the following conjecture:

Conjecture 1. *For any optimal solution of the Heilbronn triangle problem with at least nine points, each edge of the unit square $[0, 1]^2$ contains exactly two points.*

Unfortunately, we are unable to give a mathematical proof of this conjecture. Instead, using Gurobi, we prove a slightly weaker result in Proposition 5, which states that eight points are approximately located near edges in an optimal solution. Note that this result is a strengthening over Proposition 3.

Proposition 5. *Let $\varepsilon = 10^{-2}$. For the Heilbronn triangle problem with nine points, there exists an optimal solution that satisfies the following constraints:*

- (i) $y_1 = 0$ and $y_2 \leq \varepsilon$.
- (ii) $y_8 \geq 1 - \varepsilon$ and $y_9 = 1$.
- (iii) For some binary variables $c_{1i}, c_{2i} \in \{0, 1\}$, $i = 1, \dots, n$,

$$\sum_{i=1}^n c_{1i} = 2, \quad x_i \leq 1 + \varepsilon - c_{1i}, \quad i = 1, \dots, n \quad (15a)$$

$$\sum_{i=1}^n c_{2i} = 2, \quad x_i \geq c_{2i} - \varepsilon, \quad i = 1, \dots, n. \quad (15b)$$

Proof. We compare two types of solutions as below:

Solution A: Exactly eight points near edges. Let us solve a restricted version of the Heilbronn triangle problem with nine points in which we assume that Conjecture 1 holds true. In particular, we enforce the following restrictions:

$$y_1 = y_2 = 0, \quad y_8 = y_9 = 1, \quad \sum_{i=1}^9 c_{1i} = \sum_{i=1}^9 c_{2i} = 2, \quad c_{2i} \leq x_i \leq 1 - c_{1i}, \quad i = 1, \dots, 9.$$

Using Approach 1, augmented with other enhancements, Gurobi finds a feasible solution with an objective value of $L_A = \underline{H}_9 = 0.0548765299$ in 317.611 seconds.

Solution B: At most seven points near edges. Let us solve another restricted version of the Heilbronn triangle problem with nine points in which we assume at most seven points can be near edges, which is guaranteed that there exists a point whose distance to all edges is at least ε . In particular, we enforce the following restrictions for this purpose, $i = 1, \dots, n$ with $n = 9$:

$$\begin{aligned} y_1 &= 0 && \text{(at least one point on } y = 0) \\ y_9 &= 1 && \text{(at least one point on } y = 1) \\ x_i &\leq 1 - c_{1i} && \text{(forces } x_i = 0 \text{ if } c_{1i} = 1) \\ x_i &\geq c_{2i} && \text{(forces } x_i = 1 \text{ if } c_{2i} = 1) \\ x_i &\geq \varepsilon(1 - c_{1i}) && \text{(otherwise } x_i \geq \varepsilon \text{ away from } x = 0) \\ x_i &\leq 1 - \varepsilon(1 - c_{2i}) && \text{(otherwise } x_i \leq 1 - \varepsilon \text{ away from } x = 1) \\ 1 &\leq \sum_{i=1}^n c_{1i} \leq 2 && \text{(one or two points within } \varepsilon \text{ of } x = 0) \\ \sum_{i=1}^n c_{2i} &= 1. && \text{(exactly one point within } \varepsilon \text{ of } x = 1) \end{aligned}$$

Hence, at most two points may lie within ε of each of the edges $y = 0$, $y = 1$, and $x = 0$, while at most one point may lie within ε of the edge $x = 1$, giving a maximum of $2 + 2 + 2 + 1 = 7$ near-boundary points in total.

Using Approach 1, adapted with the Solution B constraints and all other proven enhancements, Gurobi is able to provide a certified upper bound of $U_B = \overline{H}_9 = 0.0546346906$ in 82 359.163 seconds.

Notice that the gap between two bounds L_A and U_B is computed as

$$|L_A - U_B| = 0.0548765299 - 0.0546346906 = 0.0002418393 > 0.$$

Hence, we deduce that a globally optimum solution of the Heilbronn triangle problem with nine points cannot be of type B. Hence, it must be of type A.

Note that no edge can host three points within distance ε : such a triple would form a triangle of area at most $\varepsilon/2 = 5 \times 10^{-3}$, far below the certified lower bounds, so the model remains valid while capping the total number of near-boundary points at seven. $\square$

Notice that equations in Item (i) (resp. Item (ii)) imply that two points must be near the edge $y = 0$ (resp. $y = 1$) while equations in Item (iii) imply that two points must be near edge $x = 0$ (equations (15a) and $x = 1$ (equations (15b)) each.

Remark 2. In the proof of Proposition 5, we also tried a lighter alternative for Solution B in which we replaced the horizontal near-edge logic by the single inequality $y_2 \geq \varepsilon$. This enforces exactly one point on $y = 0$ (since $y_1 = 0$) while keeping the same vertical near-edge controls as above, and it permits up to two points near each vertical edge and one point on $y = 1$. Although this variant is syntactically simpler, in our experiments it was less time-efficient than the Scenario B formulation stated above.

3.3 Third Group: Local Packing and Separation

3.3.1 Rectangles with at Most Two Points

The following enhancement is based on a simple geometric observation about feasible configurations of the Heilbronn triangle problem: In any optimal solution, a rectangle whose area is less than $2\underline{H}_n$ cannot contain three points, where $\underline{H}_n \leq H_n^*$. We first formalize this observation in Proposition 6, and then show how we exploit it in our formulation via a particular family of rectangles in Corollary 3.

Proposition 6. *Consider $\underline{H}_n \leq H_n^*$ and let $R \subset [0, 1]^2$ be a rectangle whose area is less than $2\underline{H}_n$. Then, in any optimal solution of the Heilbronn triangle problem with n points, R contains at most two points.*

Proof. Let $(x^*, y^*) \in [0, 1]^n \times [0, 1]^n$ be an optimal solution of the Heilbronn triangle problem with n points. Assume, for contradiction, that R contains points i, j and k . Then, the area of the triangle formed by these points is at most $\frac{1}{2} \text{area}(R) < \underline{H}_n$. However, this contradicts the optimality of (x^*, y^*) since the area of this triangle must be at least $\underline{H}_n$. $\square$

We now explain how we utilize Proposition 6 to strengthen our formulations for the Heilbronn triangle problem. In particular, we partition the unit square into horizontal strips whose height is less than $2\underline{H}_n$. Then, by the introduction of a new set of binary variables r_{ij} , we enforce that i) each strip can contain at most two points and ii) each point must be contained in exactly one strip. Figure 2 illustrates this partitioning strategy and Corollary 3 formalizes this argument.

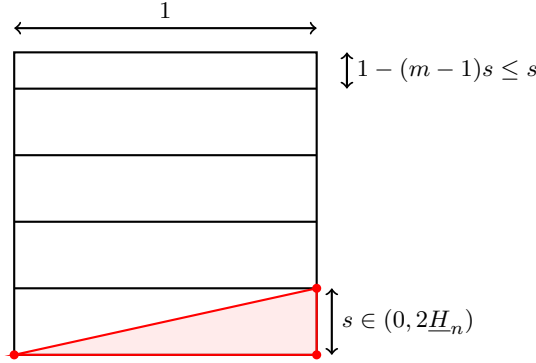


Fig. 2: The unit square partitioned into four strips of height s and a final strip of height $1 - (m-1)s < s$. Each strip has area less than $2\underline{H}_n$, so it can contain at most two points.

Corollary 3. *Consider $\underline{H}_n \leq H_n^*$ and $s \in (0, 2\underline{H}_n)$. Let m be the smallest integer such that $(m-1)s < 1 \leq ms$ and partition the unit interval $[0, 1]$ into m subintervals $[L_p, U_p]$ such that $L_p = (p-1)s$ and $U_p = \min\{1, ps\}$, for $p = 1, \dots, m$. Then, there exists an optimal solution to the Heilbronn triangle problem that satisfies the following constraints for some binary variables $r_{pi} \in \{0, 1\}$, $p = 1, \dots, m, i = 1, \dots, n$:*

- (i) $\sum_{i=1}^n r_{pi} \leq 2, \quad p = 1, \dots, m.$
- (ii) $\sum_{i=1}^n r_{pi} = 1, \quad i = 1, \dots, n.$
- (iii) $L_p - (1 - r_{pi}) \leq y_i \leq U_p + (1 - r_{pi}) \quad p = 1, \dots, m, i = 1, \dots, n.$

In our computational experiments, we instantiate Corollary 3 with the specific choice of $s = 2\underline{H}_n - 10^{-6}$. This choice guarantees that the tolerance is small enough to be negligible compared to $\underline{H}_n$, but large enough to prevent round-off errors from making the strip area equal to or slightly larger than $2\underline{H}_n$.

Remark 3. *Proposition 6 suggests that one could, in principle, strengthen Corollary 3 by introducing additional families of rectangles so as to cover more (or even all) rectangles in $[0, 1]^2$ whose area is below the threshold $2\underline{H}_n$. For example, one natural extension would be to replicate the construction of Corollary 3 with vertical strips, or to add further rotated or non-uniform rectangles, each with its own set of binary assignment variables and capacity constraints. However, when we implemented the same strip-based encoding also for vertical rectangles, we quickly observed that the additional binary variables substantially increased the model size and solution time, while providing only marginal tightening in practice. On the basis of these preliminary experiments, we therefore chose to restrict this enhancement to the horizontal strips used in Corollary 3.*

3.3.2 Small Squares Containing at Most One Point

The second enhancement is based on another geometric intuition: In any optimal solution, a square whose side is less than $\underline{H}_n$ cannot contain two points, where $\underline{H}_n \leq H_n^*$ (otherwise, those two points together with any third point in $[0, 1]^2$ would form a triangle whose area is necessarily below the given lower bound $\underline{H}_n$). We first formalize this observation in Proposition 7, and then show how we exploit it in our formulation via a particular family of squares in Corollary 4.

Proposition 7. *Consider $\underline{H}_n \leq H_n^*$ and let $R \subset [0, 1]^2$ be a square whose side is less than $\underline{H}_n$. Then, in any optimal solution of the Heilbronn triangle problem with n points, R contains at most one point.*

Proof. Let $(x^*, y^*) \in [0, 1]^n \times [0, 1]^n$ be an optimal solution of the Heilbronn triangle problem with n points. Assume, for contradiction, that R contains points i and j . Observe that the distance between these points is less than $\underline{H}_n \sqrt{2}$. Now, consider a point $(\tilde{x}, \tilde{y}) \in [0, 1]^2$. Notice that the area of triangle with corners (x_i^*, y_i^*) , (x_j^*, y_j^*) and $(\tilde{x}, \tilde{y})$ is less than $\frac{1}{2}(\underline{H}_n \sqrt{2})\sqrt{2} = \underline{H}_n$ (since the perpendicular distance of the point $(\tilde{x}, \tilde{y})$ to the line segment formed by points (x_i^*, y_i^*) and (x_j^*, y_j^*) is at most $\sqrt{2}$). However, this contradicts the optimality of (x^*, y^*) since the area of this triangle must be at least $\underline{H}_n$. $\square$

We again explain how we utilize Proposition 6 to strengthen our formulations for the Heilbronn triangle problem. In particular, we partition the unit square into a regular $m \times m$ grid of axis-aligned subsquares of side length $1/m < \underline{H}_n$. Then, by the introduction of a new set of binary variables u_{pqi} , we enforce that i) each subsquare can contain at most one point and ii) each point must be contained in exactly one subsquare. Figure 3 illustrates this partitioning strategy and Corollary 4 formalizes this argument.

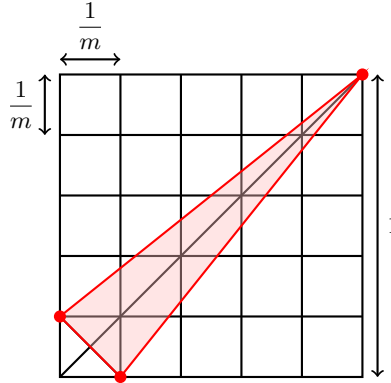


Fig. 3: Partitioning the unit square into an $m \times m$ grid. If $1/m < \underline{H}$, then each subsquare can contain at most one point.

Corollary 4. *Consider $\underline{H}_n \leq H_n^*$ and let $m \geq 1$ be an integer such that $1/m < \underline{H}_n$. Equipartition the unit interval $[0, 1]$ into m subintervals $[L_p, U_p]$ such that $L_p = \frac{p-1}{m}$ and $U_p = \frac{p}{m}$, for $p = 1, \dots, m$. Then, there exists an optimal solution to the Heilbronn triangle problem that satisfies the following constraints for some binary variables $u_{pqi} \in \{0, 1\}$, $p = 1, \dots, m$, $q = 1, \dots, m$, $i = 1, \dots, n$:*

- (i) $\sum_{i=1}^n u_{pqi} \leq 1$, $p = 1, \dots, m$, $q = 1, \dots, m$.
- (ii) $\sum_{p=1}^m \sum_{q=1}^m u_{pqi} = 1$, $i = 1, \dots, n$.
- (iii) $L_p - (1 - u_{pqi}) \leq x_i \leq U_p + (1 - u_{pqi})$, $L_q - (1 - u_{pqi}) \leq y_i \leq U_q + (1 - u_{pqi})$, $p, q = 1, \dots, m$, $i = 1, \dots, n$.

In our computational experiments, we instantiate Corollary 4 with the specific choice of $m = \left\lfloor \frac{1}{\underline{H}_n - 10^{-6}} \right\rfloor$.

Remark 4. *Proposition 7 suggests that one could, in principle, strengthen Corollary 4 by adding further families of small squares in $[0, 1]^2$, together with corresponding binary assignment variables and capacity constraints, so as to approximate the full geometric condition for all admissible squares. However, in our computational experiments (see Remark 3) we observed that each such extension substantially increases the number of binary variables and the overall model size, which in turn led to longer solution times and no commensurate practical benefit. For this reason, we restrict this enhancement to the single regular $m \times m$ grid of axis-aligned subsquares used in Corollary 4, which already leverages Proposition 7 in a numerically effective way.*

3.3.3 Applying the Heilbronn Triangle Problem in Smaller Rectangles

In Section 3.3.1, we established bounds by partitioning the square into strips small enough to contain at most two points. We now generalize this approach to larger rectangles (strips of greater height) that can

contain three or more points. The core idea is to determine the *capacity* of a horizontal strip of a given height. If we cannot place m points into a strip without forcing the minimum triangle area to fall below our lower bound of $\underline{H}_n$, then that strip is constrained to contain at most $m - 1$ points.

Since we have $y_1 \leq \dots \leq y_n$ due to Proposition 2(i), determining the capacity of a bottom strip $[0, 1] \times [0, 1/\kappa]$ imposes explicit bounds on some of the y_i variables (here, κ is the number of strips that equipartition vertical edge of the unit square). Specifically, if a strip of height $1/\kappa$ can contain at most m points, then the $(m + 1)$ -th point must lie strictly above $1/\kappa$ (i.e., $y_{m+1} \geq 1/\kappa$). By systematically applying this logic to partitions of the unit square, we derive rigorous variable bounds that significantly prune the search space *a priori*. Figure 4 illustrates the concept of solving the Heilbronn problem within such a restricted domain.

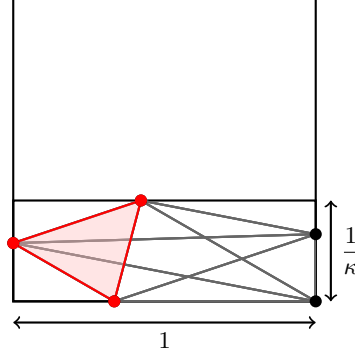


Fig. 4: Applying the Heilbronn triangle problem in smaller rectangles to derive cardinality constraints. The height of the restricted strip is $1/\kappa$, where κ is the number of strips.

Proposition 8. Fix $\kappa \geq 2$ and partition the unit square into κ equal horizontal strips of height $1/\kappa$. Let m_κ be the maximum number of points that can be placed in a strip with height $1/\kappa$ and width 1 such that the minimum area of any triangle formed by these points is at least $\underline{H}_n$. Then, there exists an optimal solution to the Heilbronn triangle problem that satisfies the following constraints:

$$y_{\ell m_\kappa + 1} \geq \frac{\ell}{\kappa} \quad \text{and} \quad y_{n - \ell m_\kappa} \leq 1 - \frac{\ell}{\kappa}, \quad \text{for } \ell = 1, \dots, \kappa - 1.$$

Proof. Let us first consider the lower bound. The first ℓ strips together form a rectangle of height ℓ/κ . If $y_{\ell m_\kappa + 1} < \ell/\kappa$, then the first $\ell m_\kappa + 1$ points lie strictly inside these first ℓ strips. By the Generalized Pigeonhole Principle, at least one of these ℓ strips must contain $\lceil (\ell m_\kappa + 1)/\ell \rceil = m_\kappa + 1$ points. However, by the definition of m_κ , placing $m_\kappa + 1$ points in a strip of height $1/\kappa$ forces the minimum triangle area to be strictly less than $\underline{H}_n$, contradicting feasibility. Thus, we must have $y_{\ell m_\kappa + 1} \geq \ell/\kappa$.

For the upper bound, consider the top ℓ strips, which cover the vertical range $[1 - \ell/\kappa, 1]$. Suppose, for the sake of contradiction, that $y_{n - \ell m_\kappa} > 1 - \ell/\kappa$. Since the y -coordinates are sorted in a non-decreasing fashion ($y_1 \leq \dots \leq y_n$), this inequality implies that all points y_i with indices $i \geq n - \ell m_\kappa$ must also lie strictly within this top region. The number of such points is $n - (n - \ell m_\kappa) + 1 = \ell m_\kappa + 1$. We are thus attempting to place $\ell m_\kappa + 1$ points into a region consisting of ℓ strips. By the Generalized Pigeonhole Principle, at least one of these top strips must contain $\lceil (\ell m_\kappa + 1)/\ell \rceil = m_\kappa + 1$ points. As established in the lower bound case, a single strip of height $1/\kappa$ cannot contain $m_\kappa + 1$ points. Since this leads to a contradiction, we deduce that $y_{n - \ell m_\kappa} \leq 1 - \ell/\kappa$. $\square$

All bounds on the ordered y -coordinates, including those implied by Proposition 3 and Proposition 8, are compiled in Table 1.

Remark 5. The specific bounds listed in Table 1 were experimentally generated using an iterative procedure. We determined the capacity m_κ for strips of height $1/\kappa$ ($\kappa = 2, 3, \dots$) by solving a sequence of restricted Heilbronn problems using the Approach 1 (MIQCP) formulation described in Section 2.2.1. To accelerate these subproblems, we enforced the symmetry breaking constraints defined in Section 3.1.2, as well as at least one point on each edge constraints detailed in Section 3.2.1. For each strip height, we incremented the number of points m until the solver proved infeasibility (i.e., the optimal value dropped below $\underline{H}_n$); the largest feasible m was recorded as m_κ .

By applying this capacity logic symmetrically—considering rectangles extending upwards from $y = 0$ and downwards from $y = 1$ —we established both lower and upper bounds. The range of indices i for which we

computed these bounds depends on the instance size. For $n = 6, 7, 8$, we targeted $i = 2, \dots, n - 1$. For $n = 9, 10$, leveraging the boundary occupancy properties defined in Proposition 5 (proven for $n = 9$) and Conjecture 1 (conjectured for $n = 10$), we restricted the search to the intermediate points $i = 3, \dots, n - 2$. For points excluded from this search, the bounds in Table 1 reflect bounds derived from Corollary 1 and Proposition 5.

This pre-computation step is highly efficient. The total wall-clock times required to generate the complete set of bounds were approximately 0.346 seconds for $n = 6$, 3.247 seconds for $n = 7$, 29.337 seconds for $n = 8$, and 35.382 seconds for $n = 9$. For $n = 10$, the time increased to 1 293.015 seconds, however, this overhead remains negligible compared to the substantial reduction in the time spent by the global solver due to these variable bounds.

	$n = 6$		$n = 7$		$n = 8$		$n = 9$		$n = 10$	
i	$\underline{y}_i$	$\overline{y}_i$	$\underline{y}_i$	$\overline{y}_i$	$\underline{y}_i$	$\overline{y}_i$	$\underline{y}_i$	$\overline{y}_i$	$\underline{y}_i$	$\overline{y}_i$
1	0	0	0	0	0	0	0	0	0	0
2	0	$\frac{1}{2}$	0	$\frac{1}{2}$	0	$\frac{1}{2}$	0	$\frac{1}{100}$	0	$\frac{1}{2}$
3	$\frac{1}{5}$	$\frac{4}{5}$	$\frac{1}{6}$	$\frac{2}{3}$	$\frac{1}{7}$	$\frac{1}{2}$	$\frac{1}{10}$	$\frac{1}{2}$	$\frac{1}{11}$	$\frac{1}{2}$
4	$\frac{1}{5}$	$\frac{4}{5}$	$\frac{1}{6}$	$\frac{5}{6}$	$\frac{1}{7}$	$\frac{2}{3}$	$\frac{1}{10}$	$\frac{2}{3}$	$\frac{1}{11}$	$\frac{1}{2}$
5	$\frac{1}{2}$	1	$\frac{1}{3}$	$\frac{5}{6}$	$\frac{1}{3}$	$\frac{6}{7}$	$\frac{1}{4}$	$\frac{3}{4}$	$\frac{1}{5}$	$\frac{2}{3}$
6	1	1	$\frac{1}{2}$	1	$\frac{1}{2}$	$\frac{6}{7}$	$\frac{1}{3}$	$\frac{9}{10}$	$\frac{1}{3}$	$\frac{4}{5}$
7	–	–	1	1	$\frac{1}{2}$	1	$\frac{1}{2}$	$\frac{9}{10}$	$\frac{1}{2}$	$\frac{10}{11}$
8	–	–	–	–	1	1	$\frac{99}{100}$	1	$\frac{1}{2}$	$\frac{10}{11}$
9	–	–	–	–	–	–	1	1	$\frac{1}{2}$	1
10	–	–	–	–	–	–	–	–	1	1

Table 1: Compilation of variable bounds for y_i $y_1 \leq \dots \leq y_n$ for $n = 6, \dots, 10$. Entries are lower bounds $\underline{y}_i$ and upper bounds $\overline{y}_i$ for each index i . The symbol – indicates that the index i does not exist for that n .

4 Computational Experiments

4.1 Computational Setup

All experiments are run on a Dell workstation fitted with two 48-core Intel Xeon Gold 6248R CPUs (3.0 GHz base clock, 96 cores in total) and 256 GB of RAM. The system operates on a 64-bit Microsoft Windows 11 system. Model development and execution are mostly carried out in Visual Studio Code, and optimization is performed with Gurobi Optimizer 11.0.1 and BARON 22.4.20.

4.2 Preliminary Experiments Setup

We evaluate the proposed methods in Section 2.2 in three stages: (i) we first identify the best baseline among the three formulations in Section 4.2.1, (ii) we then select the faster solver between Gurobi and BARON in Section 4.2.2, and (iii) we finally quantify the contribution of the enhancements developed in Section 3, as reported in Section 4.2.3, in terms of computational effort. Unless otherwise specified, all experiments are run under a uniform wall-clock time limit of one day per instance.

4.2.1 Formulation Comparison

We compare the three formulations on instances with $n \in \{3, 4, 5, 6\}$ using Gurobi under default settings, except for a one-day (i.e., a 86 400-second) time limit. For Approach 3, we use its MILP relaxation with discretization parameter $p = 10$. Proven lower bounds (LB) and upper bounds (UB) as well as and runtimes (in seconds) are summarized in Table 2.

Accuracy vs. known optima.

For $n = 3$ and $n = 4$, all three approaches recover the known global optimum 0.5, and in each case the lower and upper bounds coincide.

n	Approach 1 (MIQCP) LB–UB (Time (s))	Approach 2 (QCP) LB–UB (Time (s))	Approach 3 (MILP Rel.) LB'–UB (Time (s))
3	0.5–0.5 (0.019)	0.5–0.5 (0.016)	0.5–0.5 (0.092)
4	0.5–0.5 (0.005)	0.5–0.5 (0.031)	0.5–0.5 (0.047)
5	0.1924508–0.1924508 (4.538)	0.1924507–0.1924611 (9.632)	0.1926462–0.1926462 (27.557)
6	0.1250012–0.1250137 (468.535)	0.1250019–0.1250144 (59 920.142)	0.1252447–0.1266927 (86 400)

Table 2: Combined bounds and runtimes (seconds) for the three approaches. Note that Approach 3 is a relaxation; its objective values represent upper bounds on the true optimum. The column LB' denotes the best objective found for the *relaxed* problem, which may not be a valid lower bound for the original Heilbronn problem.

For $n = 5$, the proven global optimum is 0.1924500 [10]. Approaches 1 and 2 both reach this value to within the requested relative optimality tolerance (reporting 0.1924508 and 0.1924507, respectively). Approach 1 also certifies optimality by matching its lower and upper bounds exactly (0.1924508–0.1924508), whereas Approach 2 still reports a small residual gap. Approach 3 returns a value of 0.1926462, which is strictly larger than the true optimum. This is consistent with the fact that Approach 3 is a relaxation of the original maximization problem; therefore, it provides a valid *upper bound* on the optimal value but cannot guarantee a feasible placement (a valid lower bound) for the original problem.

For $n = 6$, where the global optimum is $1/8 = 0.125$ [10], Approaches 1 and 2 again produce lower bounds consistent with the known optimum (0.1250012 and 0.1250019, respectively). In contrast, Approach 3 provides a bound of 0.1252447. As with the $n = 5$ case, this larger value reflects the relaxation gap: Approach 3 effectively overestimates the maximum area because it relaxes the non-convex constraints. Additionally, it retains a relatively wide optimality gap (0.1252447–0.1266927) within the time limit.

Computation time.

Table 2 shows that the second approach is fastest when $n = 3$, followed by the first and (by a large margin) the third. For larger instances, the first approach becomes consistently faster: at $n = 6$ it solves in about 468.535 seconds, while the second requires 59 920.142 seconds and the third hits the one-day limit. The performance gap grows sharply with n . This widening gap indicates that, beyond very small instances, only the first approach remains computationally practical, while the others become prohibitively expensive and are no longer reasonable candidates for large n .

Selection.

Both the first and second approaches produce objective values that agree with the known global optima for $n = 3, \dots, 6$, and in particular neither shows meaningful numerical deviation at the reported tolerance. Moreover, the first approach is often able to certify optimality outright by closing the lower/upper bound gap, whereas the second approach may still report a small residual gap. When runtime is taken into account, the distinction becomes sharper: although the second approach is competitive for very small instances, it becomes dramatically slower as n increases, while the first approach remains practical. The third approach is both less accurate (e.g., for $n = 5, 6$) and substantially slower. On this basis, we select Approach 1 as the baseline for all subsequent experiments.

4.2.2 Solver Comparison

We evaluate the *superior model* identified in Section 4.2.1 (without enhancements) on both BARON and Gurobi to select the best performing solver for subsequent experiments. Unless stated otherwise, both solvers are run under their default settings, except that a wall-clock limit of one day per instance is imposed. The lower and upper bounds obtained, and runtimes (in seconds) are summarized in Table 3.

Accuracy vs. known optima.

For $n \in \{3, 4\}$, both solvers recover the known global optimum 0.5, and in each case the lower and upper bounds coincide. For $n = 5$, the proven optimum is $H_5^* = \sqrt{3}/9 = 0.192450$ [10]. Gurobi attains this value

n	Gurobi		BARON	
	LB-UB	Time(s)	LB-UB	Time(s)
3	0.5–0.5	0.021	0.5–0.5	0.064
4	0.5–0.5	0.013	0.5–0.5	0.062
5	0.1924508–0.1924508	4.540	0.1924553–0.1924562	285.302
6	0.1250012–0.1250137	468.535	0.1250081–0.1992033	86 400

Table 3: Combined solver bounds and runtimes (seconds). The value 86 400 s indicates that the one-day time limit was reached.

within the requested relative optimality tolerance and certifies it by reporting matching bounds 0.1924508–0.1924508. BARON returns a slightly higher value, with bounds 0.1924553–0.1924562, which is above the proven optimum and does not close the optimality gap. For $n = 6$, where the exact value is $H_6^* = 1/8 = 0.125$ [10, 24], Gurobi reports bounds 0.1250012–0.1250137, which are consistent with the known optimum at the stated tolerance whereas BARON produces a much wider interval of 0.1250081–0.1992033.

Computation time.

For $n \in \{3, 4\}$, the runtime difference is negligible. Starting at $n = 5$, Gurobi is substantially faster (about 4.54 s vs. 285.30 s), and at $n = 6$ the gap widens dramatically: Gurobi solves in 468.535 s, while BARON reaches the one-day cap (reported as 86 400 s) without certification.

Selection.

Both solvers reproduce the known optimal value for very small instances, but starting at $n = 5$ their behavior diverges. Gurobi continues to match the known optimum to within the requested tolerance and is able to certify that value by closing the bound gap, while BARON begins to report values that exceed the proven optimum and retains a non-negligible gap. By $n = 6$, Gurobi remains consistent with the exact value $1/8$, whereas BARON no longer delivers a tight bound. Taken together with the runtime trends in Table 3, these results indicate that Gurobi is the more dependable solver for our purposes, and we therefore use it for all subsequent computations.

4.2.3 Enhancement Comparison

Building on the best approach identified in Section 4.2.1 and the solver choice in Section 4.2.2, we quantify the contribution of three enhancement groups via a full-factorial design with 2^3 experiments in Table 4.

For $n \in \{7, 8, 9\}$, Table 4 reports eight configurations corresponding to all on/off patterns of the three groups $\mathcal{G}_1, \mathcal{G}_2, \mathcal{G}_3$: (i) none, (ii–iv) singletons, (v–vii) all pairwise combinations, and (viii) all three enabled. *The groups themselves are defined uniformly for all n* ; however, each group contains several constraints whose applicability depends on the number of points n . Thus, when a group is “on” at a given n , only those constraints within the group whose eligibility conditions are satisfied are activated, while the inapplicable members of the group remain inactive.

Each configuration is evaluated by (a) wall-clock time (seconds) and (b) solution quality summarized by LB/UB (and the implied optimality gap). All runs use the default Gurobi settings and a uniform wall-clock limit per instance (entries equal to the cap in Table 4 indicate runs that timed out).

This design allows us to estimate the *main effects* of each group (marginal improvement when toggled on) and the *interaction effects* among groups across $n = 7, 8, 9$.

Accuracy vs. known optima.

For $n = 7$ and $n = 8$, the optimum values are approximately $H_7^* = 0.083859$ [11] and $H_8^* = 0.0723764$ [12]. In Table 4, some enhancement settings reproduce these known optima and certify them by reporting matching lower and upper bounds (LB=UB), while others do not. For $n = 7$, every configuration except the baseline with no enhancements (--) and the standalone group $\mathcal{G}_3$ achieves the lower bound being equal to the upper bound at H_7^* , i.e., certifies optimality. For $n = 8$, certification occurs for settings that include $\mathcal{G}_1$ (namely $\mathcal{G}_1$, $\mathcal{G}_{1,2}$, $\mathcal{G}_{1,3}$, and $\mathcal{G}_{1,2,3}$) and also for $\mathcal{G}_{2,3}$; in contrast, --, $\mathcal{G}_2$, and $\mathcal{G}_3$ do not certify.

For $n = 9$, the exact optimum is not known; the best published interval is $0.054875999 < H_9^* < 0.054878314$ [15]. Among our configurations, only $\mathcal{G}_{1,2}$ and $\mathcal{G}_{1,2,3}$ return the lower bound equals to the upper bound at values consistent with that interval, thereby producing certificates. All other settings (--, $\mathcal{G}_1$, $\mathcal{G}_2$, $\mathcal{G}_3$, $\mathcal{G}_{1,3}$, $\mathcal{G}_{2,3}$) fail to certify within the allowed time (see Table 4).

¹For the $n = 9$ instance, a one-day preliminary computation is required to establish Proposition 5; thereafter, combinations containing enhancement group 2 could be used.

n	Groups	Lower Bound–Upper Bound	Time(s)
7	–	0.0838601–0.1599112	86 400
	1	0.0838597–0.0838597	16.570
	2	0.0838599–0.0838599	218.782
	3	0.0838598–0.1229744	86 400
	1, 2	0.0838601–0.0838601	7.537
	1, 3	0.0838599–0.0838599	29.638
	2, 3	0.0838600–0.0838600	104.361
	1, 2, 3	0.0838595–0.0838595	4.542
8	–	0.0723771–0.1684433	86 400
	1	0.0723774–0.0723774	314.720
	2	0.0723775–0.0921830	86 400
	3	0.0723772–0.1475525	86 400
	1, 2	0.0723773–0.0723773	385.604
	1, 3	0.0723770–0.0723770	82.888
	2, 3	0.0723775–0.0723775	3 639.555
	1, 2, 3	0.0723767–0.0723767	53.334
9 ¹	–	–	86 400
	1	0.0548767–0.0640982	86 400
	2	0.0548770–0.0893424	86 400
	3	0.0483269–0.1803900	86 400
	1, 2	0.0548769–0.0548769	3 334.102
	1, 3	0.0548765–0.0689799	86 400
	2, 3	0.0548771–0.0730435	86 400
	1, 2, 3	0.0548767–0.0548767	633.331

Table 4: Combined LB/UB and runtimes (seconds) for enhancement groups across $n \in \{7, 8, 9\}$, computed with Gurobi. An entry of 86 400 s indicates the time cap was reached.

Computation time.

The baseline -- and the singleton $\mathcal{G}_3$ consistently time out (86 400 s). Among certifying configurations, there is a clear positive interaction: enabling all three groups $\mathcal{G}_{1,2,3}$ is fastest for each n (4.542 s for $n = 7$; 53.334 s for $n = 8$; 633.331 s for $n = 9$), improving substantially over strong pairwise settings (e.g., $\mathcal{G}_{1,2}$: 7.537 s, 385.604 s, and 3 334.102 s, respectively). Some combinations can certify but remain slow (e.g., $\mathcal{G}_{2,3}$ at $n = 8$: 3 639.555 s, indicating that $\mathcal{G}_3$ is beneficial only when paired with $\mathcal{G}_1$ and/or $\mathcal{G}_2$, and otherwise may hinder pruning.

Selection.

Across $n = 7$ and $n = 8$, the enhancement settings that include $\mathcal{G}_1$ are consistently able to match and certify the known global optima, while configurations that exclude $\mathcal{G}_1$ often fail to do so. For $n = 9$, where only a bracket is available, the only settings that both respect this interval and close the gap are $\mathcal{G}_{1,2}$ and $\mathcal{G}_{1,2,3}$. When runtime is also taken into account (Table 4), $\mathcal{G}_{1,2,3}$ not only certifies whenever certification is possible, but does so more efficiently than the other certifying combinations. We therefore adopt $\mathcal{G}_{1,2,3}$ as the default configuration for subsequent experiments.

4.3 Final Results

Guided by the results above, we adopt Approach 1 as the base formulation, use Gurobi as the solver, and start from the full enhancement bundle $\mathcal{G}_{1,2,3}$ (the best-performing group combination). Instance-specific enhancements are activated only when valid for the given n : the objective-value bound in Section 3.1.1 is inapplicable at $n = 3$; the one-point-on-each-edge result (Proposition 3) also does not apply at $n = 3$; the two-points-per-edge motif (Proposition 3.2.2) is used only for $n \in \{7, 8\}$; and the near-boundary pattern for $n = 9$ (Section 3.2.3) is used only at $n = 9$. The performance of the resulting setting for $3 \leq n \leq 10$ is summarized in Table 5, which reports, for each n , the certified lower/upper bounds, the wall-clock time, and the *computed area*, which is computed directly from the reported coordinates of an optimal placement.

Since the optimal values for $n = 5, 6, 7, 8$ are already established in the literature, our figures for these rows essentially coincide with the known values; any tiny discrepancies stem from numerical tolerances in the solver.

The only previous record for the $n = 9$ case is due to Chen et al., who reported $0.054875999 < H_9^* < 0.054878314$ in [15]. Their result was the product of an intensive grid search method run on a

specialized, large-scale computing infrastructure, including an IBM NUMA cluster with 4TB of RAM and three GPGPU machines equipped with 12 NVIDIA Tesla C2050 cards in total. This entire search required 2695833.174 seconds (approximately 31 days) of GPGPU computation time [15]. In stark contrast, our refined mathematical programming model achieves a certified global optimum (as shown in Table 5) in just 633.331 seconds (approximately 10 minutes) when running on a single CPU-based workstation. We would like to remind the reader that even if we include the pre-processing time needed to numerically prove Proposition 5 (also see, Footnote 1), the resulting computational effort is still significantly smaller compared to reference [15].

n	Lower Bound	Upper Bound	Time (s)	Computed Area ²	Proven Answer
3	0.5000000	0.5000000	0.016	0.5000000	0.5
4	0.5000000	0.5000000	0.026	0.5000000	0.5
5	0.1924506	0.1924506	0.186	0.1924499	0.1924500
6	0.1250010	0.1250010	0.787	0.1249999	0.125
7	0.0838594	0.0838594	5.280	0.0838584	0.0838591
8	0.0723767	0.0723767	53.334	0.0723758	0.0723764
9	0.0548767	0.0548767	633.331	0.0548756	-
10	0.0465383	-	86 400	0.0465369	-

Table 5: Optimization results for varying n values.

The last row of Table 5 shows our best result for $n = 10$ run under a *restricted model* that included two strong, but unproven structural conjectures: (i) We assume that Conjecture 1 holds true, that is, an optimal solution must place *exactly* eight points on the boundary edge. This hypothesis is motivated by the two-point-per-edge *near-boundary* pattern observed for $n = 9$ in Section 3.2.3. (ii) We add the symmetry-breaking constraint $y_5 \leq \frac{1}{2}$, justified by the horizontal partition bounds in Table 1 that limit how many points can lie in each half. Under this restricted model, $n = 10$ quickly exposed our current hardware ceiling. Across multiple runs with progressively tighter cuts, the solver consistently certified a *provable* lower bound of 0.0465383 (with a computed area of 0.0465369) in about 800 seconds—already matching the best empirical value of 0.046537 reported by Comellas and Yebra in [8]. Closing the remaining gap to obtain a matching upper bound (and thus a full certificate of optimality) appears to require substantially longer wall time or new structural insights. While $n = 10$ remains open, the speed at which such a strong lower bound is achieved highlights the promise of the strengthened formulation once additional computational power or stronger valid inequalities are available.

The best configurations found for $n = 6$ through $n = 10$ are visualized in Figure 5. The complete point coordinates are also provided in Table A1 of Appendix A.

5 Conclusions

In this paper, we explored three novel modeling approaches for the Heilbronn triangle problem and systematically strengthened each one with several groups of enhancements. Every group was first tested on small instances, both on its own and in combination with the others, and only the combinations that consistently reduced run-time while preserving a proof of optimality were retained.

After experimenting with various modeling approaches, we also benchmarked alternative solvers. In particular, a BARON implementation was tested against a Gurobi setup; the comparison showed that Gurobi consistently delivered shorter runtimes and tighter bounds. With the optimal combination of constraints and the more efficient solver in place, the results were consolidated into Table 5.

With this configuration in place, we were able to *certify* the global optimum for the nine-point case: the solver returned matching bounds (LB = UB) at 0.0548767 for $n = 9$, thereby closing the previously published bracket from [15] and providing a full certificate of optimality (see Table 5). For $n = 10$, full certification remains open: under a restricted model that imposes two unproven structural assumptions—(i) the “two points per edge” near-boundary pattern and (ii) the symmetry-breaking constraint $y_5 \leq \frac{1}{2}$ —we obtained a strong valid lower bound of 0.0465383 (with a computed area of 0.0465369) in about 800 seconds; however, no matching upper bound was reached within the one-day limit. Further structural insights or longer runs will likely be required to settle $n = 10$.

Looking ahead, we see a concrete path to certification for larger instances. First, the structural phenomenon behind Proposition 3.2.3 (“approximately two points per edge”) appears to extend to $n = 10$; a

²The solver-reported objective value is subject to the solver’s numerical tolerances and model approximations. To obtain a more accurate value for the specific configuration shown in Table A1, we recomputed the minimum triangle area directly from the listed coordinates and report it here as the *computed area*.

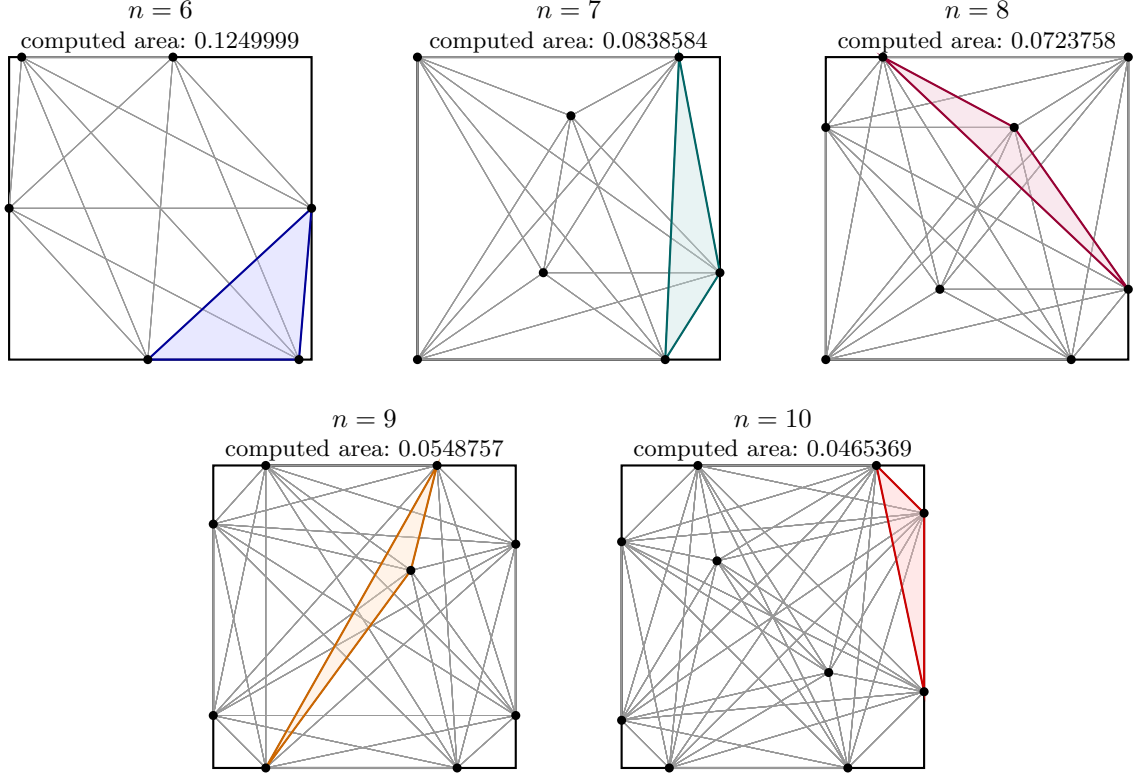


Fig. 5: Point placements for $n = 6$ to $n = 10$. One of the smallest triangles is highlighted.

formal proof of this near-boundary pattern might materially shrink the feasible region and could close the remaining gap. Second, we have developed additional symmetry-breaking rules that, while not yet proved, consistently tighten relaxations in practice and are strong candidates for formal validation. Third, we plan to augment the model with *orientation-count* constraints: letting $\beta_{ijk} \in \{0, 1\}$ indicate the sign of the signed area for triangle (i, j, k) , one can bound the total number of “positive” vs. “negative” triangles, thereby restricting combinatorial degeneracies that currently inflate the search space. Together with tighter versions of existing cuts, these directions make the framework technically promising for certifying $n = 10$ and pushing toward $n = 11$.

Declarations

Ethics approval and consent to participate

Not applicable

Consent for publication

Not applicable

Funding

Not applicable

Availability of data and materials

Data is provided within the manuscript and references therein.

Competing interests

The authors declare that they have no competing interests.

Authors' contributions

Amirali Modir and Amirhossein Monji worked on the methodology, implementation, visualization, analysis, writing and Burak Kocuk worked on the methodology, analysis, writing.

References

- [1] Roth, K.F.: On a problem of heilbronn. *Journal of the London Mathematical Society* **1**(3), 198–204 (1951)
- [2] Roth, K.F.: On a problem of heilbronn, ii. *Proceedings of the London Mathematical Society* **3**(2), 193–212 (1972)
- [3] Roth, K.F.: On a problem of heilbronn, iii. *Proceedings of the London Mathematical Society* **3**(3), 543–549 (1972)
- [4] Roth, K.F.: Developments in heilbronn’s triangle problem. *Advances in Mathematics* **22**(3), 364–385 (1976)
- [5] Komlós, J., Pintz, J., Szemerédi, E.: On heilbronn’s triangle problem. *Journal of the London Mathematical Society* **2**(3), 385–396 (1981)
- [6] Komlós, J., Pintz, J., Szemerédi, E.: A lower bound for heilbronn’s problem. *Journal of the London Mathematical Society* **2**(1), 13–24 (1982)
- [7] Goldberg, M.: Maximizing the smallest triangle made by n points in a square. *Mathematics Magazine* **45**(3), 135–144 (1972)
- [8] Comellas, F., Yebra, J.L.A.: New lower bounds for heilbronn numbers. *the electronic journal of combinatorics*, 6–6 (2002)
- [9] Bertram–Kretzberg, C., Hofmeister, T., Lefmann, H.: An algorithm for heilbronn’s problem. *SIAM Journal on Computing* **30**(2), 383–390 (2000)
- [10] Lu, Y., Jingzhong, Z., Zhenbing, Z.: On Goldberg’s Conjecture: Computing the First Several Heilbronn Numbers. Universität Bielefeld. SFB 343. Diskrete Strukturen in der Mathematik, ??? (1991)
- [11] Zeng, Z., Chen, L.: On the heilbronn optimal configuration of seven points in the square. In: *International Workshop on Automated Deduction in Geometry*, pp. 196–224 (2008). Springer
- [12] Dehbi, L., Zeng, Z.: Heilbronn’s problem of eight points in the square. *Journal of Systems Science and Complexity* **35**(6), 2452–2480 (2022)
- [13] Jiang, T., Li, M., Vitányi, P.: The expected size of heilbronn’s triangles. In: *Proceedings. Fourteenth Annual IEEE Conference on Computational Complexity (Formerly: Structure in Complexity Theory Conference)*(Cat. No. 99CB36317), pp. 105–113 (1999). IEEE
- [14] Jiang, T., Li, M., Vitányi, P.: The average-case area of heilbronn-type triangles. *Random Structures & Algorithms* **20**(2), 206–219 (2002)
- [15] Chen, L., Xu, Y., Zeng, Z.: Searching approximate global optimal heilbronn configurations of nine points in the unit square via gpgpu computing. *Journal of Global Optimization* **68**, 147–167 (2017)
- [16] Gurobi Optimization, L.: Gurobi Optimizer Reference Manual. www.gurobi.com/documentation/ (2022)
- [17] Tawarmalani, M., Sahinidis, N.V.: A polyhedral branch-and-cut approach to global optimization. *Mathematical Programming* **103**(2), 225–249 (2005)
- [18] McCormick, G.P.: Computability of global solutions to factorable nonconvex programs: Part i —convex underestimating problems. *Mathematical Programming* **10**(1), 147–175 (1976)
- [19] Dey, S.S., Gupte, A.: Analysis of milp techniques for the pooling problem. *Operations Research* **63**(2), 412–427 (2015)
- [20] Dey, S.S., Kocuk, B., Santana, A.: Convexifications of rank-one-based substructures in qcqps and applications to the pooling problem. *Journal of Global Optimization* **77**(2), 227–272 (2020)

- [21] Jalilian, M., Kocuk, B.: Improved rank-one-based relaxations and bound tightening techniques for the pooling problem. *Optimization and Engineering*, 1–51 (2025)
- [22] Litvinchev, I., Infante, L., Ozuna Espinosa, E.: Approximate Packing: Integer Programming Models, Valid Inequalities and Nesting, vol. 105, pp. 117–135. Springer, ??? (2015). https://doi.org/10.1007/978-3-319-18899-7_9
- [23] Taspinar, R., Kocuk, B.: Discretization-based solution approaches for the circle packing problem. *Engineering Optimization* **56**(12), 2060–2077 (2024)
- [24] Dress, A.W.M., Yang, L., Zeng, Z.: Heilbronn problem for six points in a planar convex body. In: *Minimax and Applications*, pp. 173–190. Springer, Boston, MA, USA (1995)

Appendix A Coordinates for $n = 6$ to $n = 10$

All coordinates lie in the unit square $[0, 1]^2$. For each n , the table lists the point coordinates $\{(x_i, y_i)\}_{i=1}^n$. For each configuration, we also report the value H_n , computed directly from those coordinates as the minimum area among all $\binom{n}{3}$ triangles determined by the points. In other words, H_n is the smallest triangle area achieved by the explicit placement shown in the table.

	$n = 6$ (0.1249999)		$n = 7$ (0.0838584)		$n = 8$ (0.0723758)		$n = 9$ (0.0548757)		$n = 10$ (0.0465369)	
i	x_i	y_i	x_i	y_i	x_i	y_i	x_i	y_i	x_i	y_i
1	0.0000000	0.5002079	0.0000000	0.0000000	0.0000000	0.0000000	0.1734434	0.0000000	0.1576891	0.0000000
2	0.9583795	0.0000095	0.8191741	0.0000000	0.8114203	0.0000000	0.8062261	0.0000000	0.7479323	0.0000000
3	1.0000000	0.5002258	0.4161417	0.2872579	1.0000000	0.2324082	1.0000000	0.1734440	0.0000000	0.1576884
4	0.4585968	0.0000000	1.0000000	0.2872583	0.3771617	0.2324082	0.0000000	0.1734439	1.0000000	0.2520800
5	0.0415842	0.9999909	0.5074140	0.8060633	0.0000000	0.7675903	0.6531127	0.6531128	0.6846219	0.3153852
6	0.5418013	1.0000000	0.8648099	1.0000000	0.6228391	0.7675912	1.0000000	0.7398341	0.3153851	0.6846210
7	–	–	0.0000000	1.0000000	0.1885812	1.0000000	0.0000000	0.8062269	0.0000000	0.7479305
8	–	–	–	–	1.0000000	1.0000000	0.1734429	1.0000000	1.0000000	0.8423081
9	–	–	–	–	–	–	0.7398355	1.0000000	0.8423085	1.0000000
10	–	–	–	–	–	–	–	–	0.2520782	1.0000000

Table A1: The exact coordinates of the point placements and the corresponding minimum area from Figure 5 for different n values.